

Hardware Accelerator for Memory-Intensive Applications in Processors

Review 2 — Proposed Solution, Architecture & Implementation Plan

Team Members

Sanshey (5451), Ayush Bajetha (5414), Keshav Dangi (5424)

Project Guide

Dr. Sushma Patil

Department

Electronics and Telecommunications

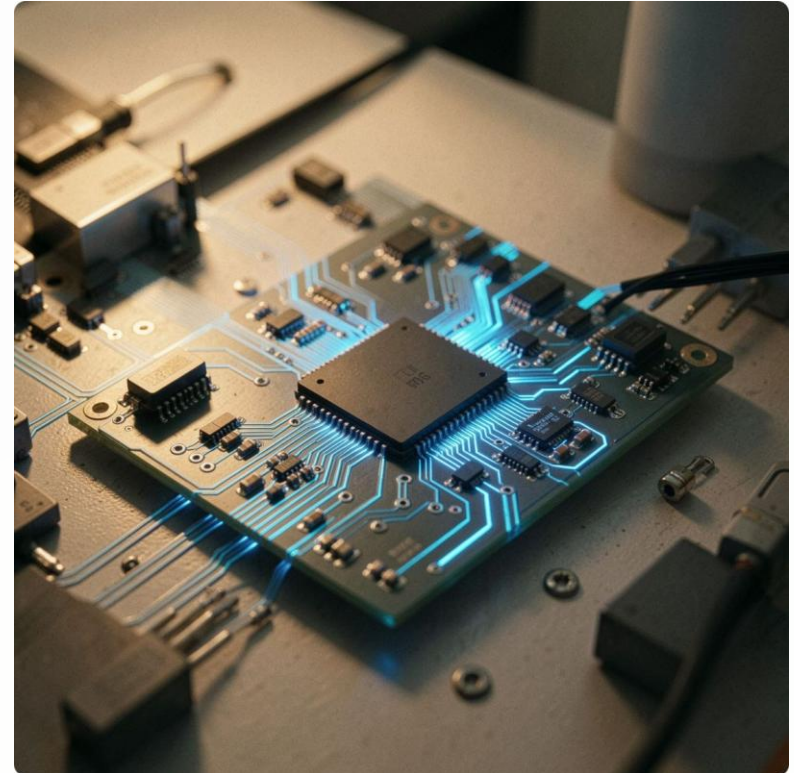
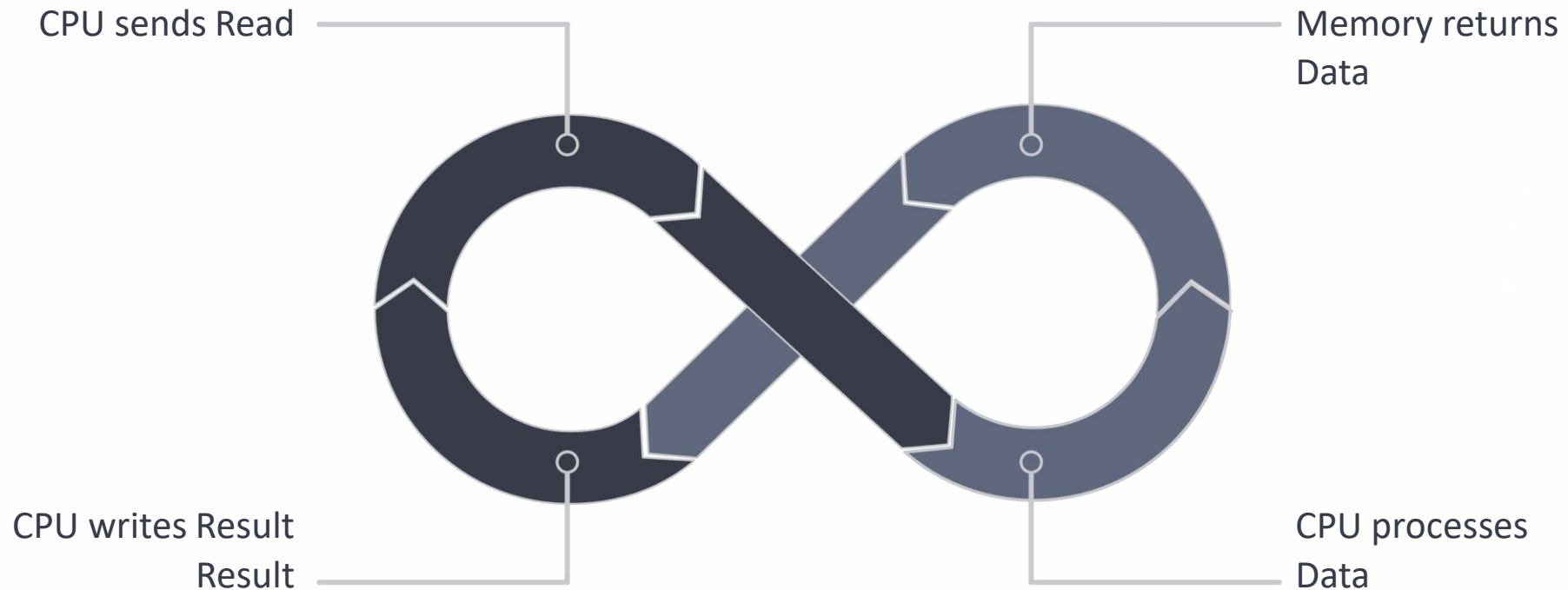


Table of Contents

1. The Memory Bottleneck
2. GPU vs TPU vs FPGA
3. Why Choose FPGA?
4. Proposed Solution
5. System Architecture
6. Hardware Components
7. Software Components
8. Working of the System
9. CPU-Only vs. Accelerator-Based Implementation
10. Implementation & Future Scope
11. Conclusion

Problem Statement: The Memory Bottleneck

In conventional processors, the CPU is significantly faster than main memory. As datasets grow, frequent memory access causes stalls, repeated data movement, and increased CPU workload — limiting overall system performance.



This repeated Read → Process → Write cycle is the core inefficiency our accelerator aims to address.

GPU vs TPU vs FPGA

All three can be used as hardware accelerators, but they are optimized for different types of workloads.

GPU

Primary purpose: graphics and parallel compute

Best for: flexible high-throughput workloads

Key strength: very high parallelism

Suitable for your project: good if flexibility and throughput matter most



TPU

Primary purpose: machine learning and tensor operations

Best for: training and inference of deep learning models

Key strength: optimized for ML dataflows

Suitable for your project: good if the workload is mostly neural-network focused



FPGA

Primary purpose: custom digital circuits and specialized acceleration

Best for: low-latency, deterministic custom hardware

Key strength: reconfigurable and hardware-tailored

Suitable for your project: best fit for memory-intensive operations with low latency, reduced data movement, and controlled power/area



Why Choose FPGA?

Custom Memory Architecture

Design scratchpad, DMA engine, and data paths.

Reduce memory stalls.

Parallelism & Predictable Timing

Custom pipelines for concurrent processing.

Controlled latency and throughput measurement.

Reprogrammability

Modify buffer size and processing units.

Flexible for iterative development.

Research Platform

Compare different architectures.

Measure actual performance metrics.

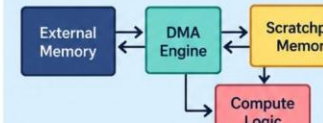
Why FPGA Suits Memory-Bottleneck Projects?

Flexible hardware to design, test and optimize memory-centric solutions

1

Custom Memory Architecture

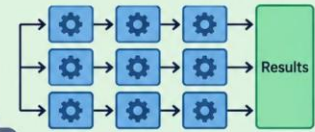
Design scratchpad, DMA and data paths tailored to your application to reduce memory stalls.



2

Parallelism & Predictable Timing

Implement custom pipelines to process multiple data elements concurrently with predictable latency and throughput.



3

Reprogrammability

Modify buffer size, number of processing units and memory access logic after testing. Ideal for projects with evolving requirements.



Design → Test → Modify → Repeat

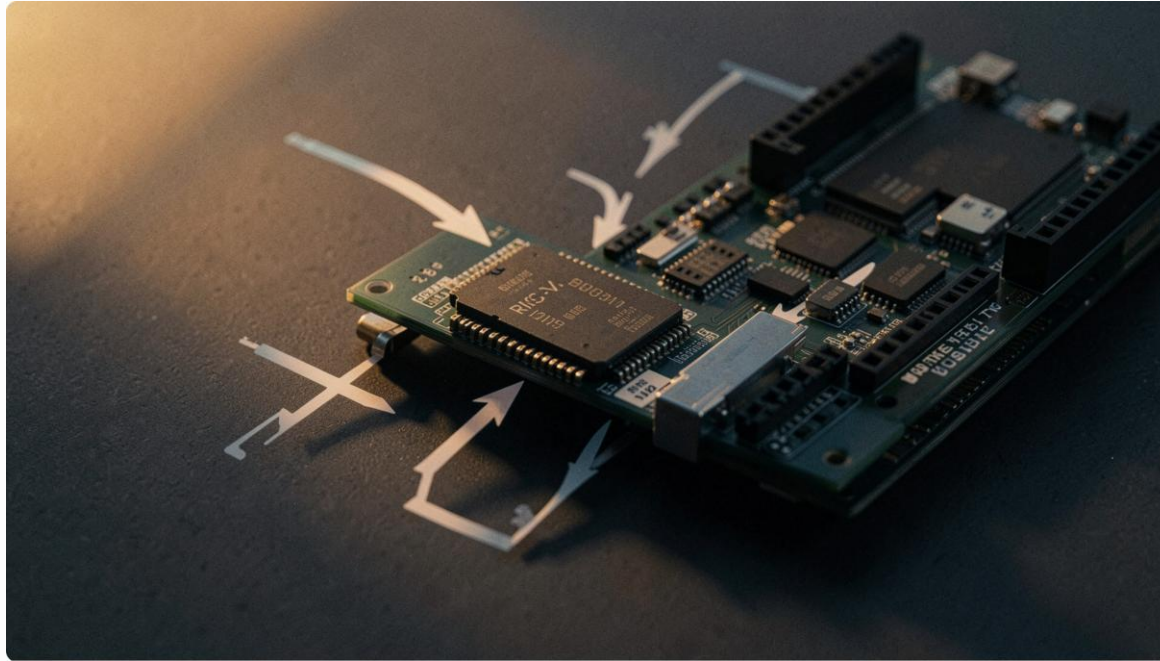
4

Good Platform for Research

Compare different architectures, measure real latency and bandwidth, and evaluate design changes for data-intensive applications.



Proposed Solution: Hardware-Software Co-Design



Our solution partitions the workload: the **RISC-V CPU** handles application logic and control, while a **hardware accelerator** executes selected memory-operations.

→ CPU retains control
Manages application flow and configures the accelerator

→ Accelerator handles computation
Offloads repetitive, memory-heavy operations

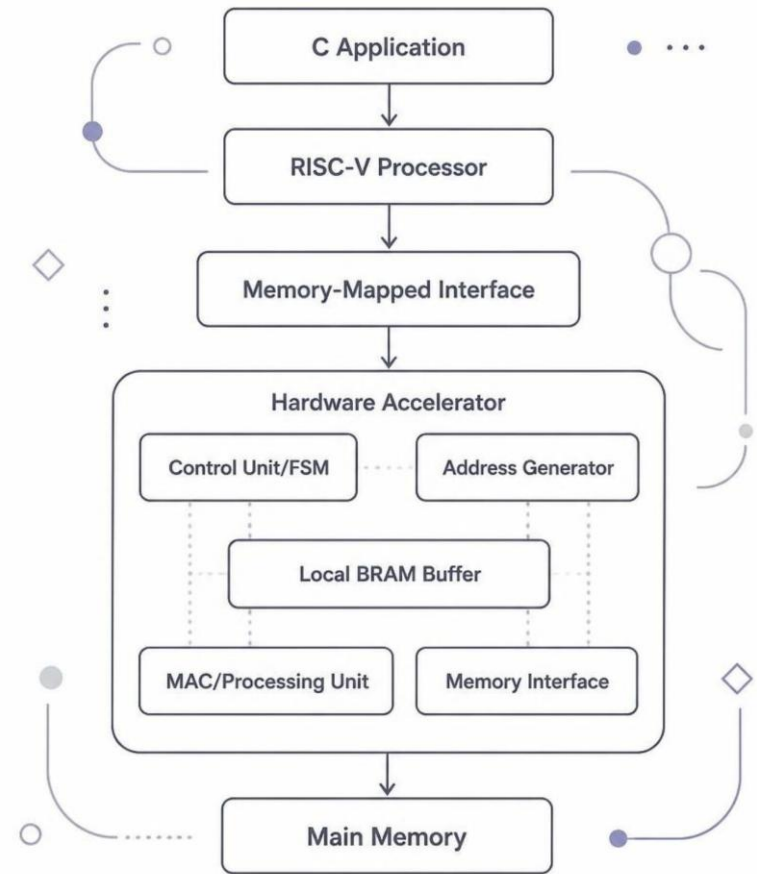
→ Local BRAM buffer
Reduces repeated main memory accesses

→ Memory-mapped interface
Simple register-based communication between CPU and accelerator

System Architecture

The architecture follows a clean pipeline: the C application runs on the RISC-V processor, which communicates with the hardware accelerator through a **memory-mapped interface**. The accelerator fetches data from main memory, processes it locally, and writes results back.

□ Data flow: Application → RISC-V CPU → Memory-Mapped Interface → Hardware Accelerator → Main Memory



Hardware Components

1	Control Unit (FSM) Manages accelerator states — IDLE, LOAD, COMPUTE, STORE, DONE
2	Address Generator Automatically generates sequential memory read/write addresses
3	Local BRAM Buffer On-chip memory storing frequently reused data to reduce latency
4	MAC / Processing Element Performs multiply-accumulate operations for matrix/vector workloads
5	Memory Interface Manages data transfer between accelerator and main memory
6	Status / Control Registers Memory-mapped registers allowing CPU to configure and monitor accelerator

Software Components



The software stack runs on the RISC-V processor and provides a clean interface to the hardware accelerator. A **C/C++ application** serves as the primary workload, with matrix multiplication or vector processing as the demonstration kernel.

01

C Application

Main program and CPU reference implementation

03

Memory-Mapped Register Control

Writes input address, size, and operation code

02

Accelerator Driver

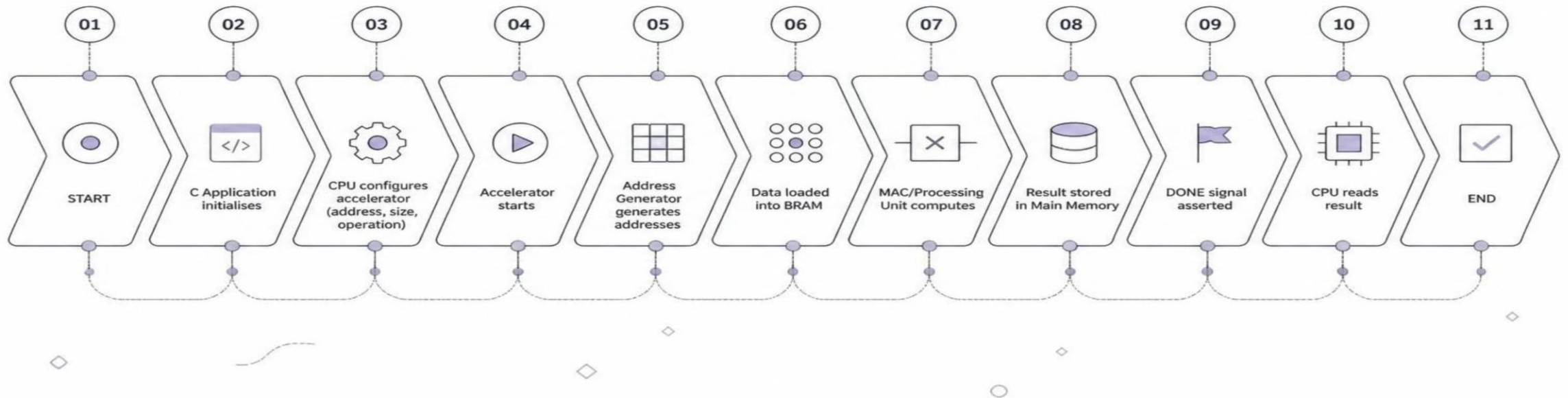
Configures registers, triggers operations, polls status

04

Benchmark & Verification

Compares accelerator output against CPU reference

Working of the System



The CPU initiates the operation by writing to control registers. The accelerator then autonomously manages data movement and computation, signalling completion via a DONE flag — allowing the CPU to continue other tasks during processing.

CPU-Only vs. Accelerator-Based Implementation

Expected qualitative differences between the two approaches. **Actual values will be measured during FPGA testing.**

Parameter	CPU-Only	Accelerator-Based
CPU Workload	Higher — handles all computation	Reduced — offloads intensive ops
Memory Management	Frequent, software-controlled	Hardware-managed, automated
Execution Cycles	Higher	Expected lower (to be measured)
Execution Time	Higher	Expected reduced (to be measured)
Throughput	Lower	Expected improved (to be measured)
Hardware Resources	Minimal — CPU only	Additional FPGA logic + BRAM

Implementation & Future Scope

Current Implementation

- **Verilog / SystemVerilog** for accelerator RTL design
- **C language** for software and driver
- **RISC-V processor** as host CPU
- **FPGA** target for synthesis and testing
- **BRAM** for on-chip local buffering
- Simulation and synthesis using standard FPGA EDA tools

Future Scope

- **DMA Support** — enable direct memory access without CPU intervention
- **Multiple Processing Elements** — parallelise computation further
- **Cache Integration** — reduce memory latency for irregular access patterns
- **Hardware Prefetching** — anticipate and fetch data before it is needed
- **Extended Workloads** — support convolution, FFT, and other memory-intensive kernels

Conclusion

-  Memory access is a key bottleneck
Large-data applications suffer from repeated CPU-memory transfers
-  Dedicated hardware offloads the CPU
Selected operations are handled by a custom accelerator
-  Local BRAM and address generation improve efficiency
On-chip buffering reduces main memory traffic
-  Software controls hardware via memory-mapped registers
Clean, low-overhead CPU–accelerator interface
-  Performance to be evaluated experimentally
CPU-only and accelerator results will be compared on FPGA

Thank You